Best Practices for Git Commit Messages

Overview

Well-written Git commit messages make it easier to review changes, debug regressions, generate changelogs, and understand the history of a codebase over time. Poor messages slow down code review and make it harder for future maintainers (including the original author) to reconstruct why a change was made.[1][2][3][4]

Core Principles

- Treat each commit as a small, coherent unit of change ("atomic commits"), ideally focused on a single purpose such as fixing one bug or adding one feature.[5][1]
- Write messages for the future reader: someone scanning `git log` months later to understand what changed and why, not just for yourself in the moment.[2][4]
- Focus the message on *what* changed and *why*, rather than *how*, since the code itself already shows the implementation details.[6][3][^2]

Recommended Structure

Most guidelines recommend a structured format with a short summary line and an optional, more detailed body.[7][8][^2]

```
<subject line>

<body (optional)>

<footer (optional: issue IDs, breaking changes, etc.)
```

- The first line is a concise subject (title) describing the change.[8][2]
- A blank line separates the subject from the body; many Git tools rely on this separation to display logs correctly.[2][7]
- The body can contain paragraphs or bullet points explaining context, rationale, and side effects, wrapped at around 72 characters per line for readability in terminals.[6][7][^2]
- The footer is often used for issue tracker references (for example, `Resolves: #123`) or breaking-change notes.[9][2]

Subject Line Rules

Commonly accepted rules for the subject line include:[7][8][2][6]

- Limit length to about 50 characters so it is not truncated in tools and on narrow displays.[1][6]

- Use the imperative mood, as if completing the sentence "If applied, this commit will …" (for example, `Fix login bug`, `Add user audit log`).[10][8][^2]
- Capitalize the first letter of the subject, but otherwise avoid title case unless your project prefers it.[6][7]
- Do not end the subject line with a period, keeping it as a short label rather than a sentence.[11][7][^6]

Body Content Guidelines

The body is optional for trivial changes but strongly recommended for anything non-obvious.[4][2]

- Explain *why* the change is needed, what problem it solves, and any trade-offs or side effects; this is often more valuable than restating what the diff already shows.[3][4][^2]
- Include relevant background, such as links to design documents, tickets, or prior decisions, so that the context is preserved with the change.[4][2]
- Use paragraphs and bullet points, wrapped at roughly 72 characters per line to keep logs readable in text-based tools.[2][7][^6]
- When referencing issues, use a consistent pattern such as `Resolves: #123` or `See also: #456` at the bottom of the message.[9][2]

Style and Formatting Conventions

Across multiple style guides, a small set of formatting conventions appears repeatedly.[3][7][2][6]

- Subject and body
  - Separate them with exactly one blank line.[7][2]
  - Keep the subject around 50 characters and body lines around 72 characters.[1][2][^7]
- Tone and tense
  - Use imperative mood verbs: `Add`, `Fix`, `Refactor`, not `Added`, `Fixing`, or `Refactored`.[10][5][^2]
  - Keep the tone neutral and specific, avoiding vague phrases like `More fixes` or `Stuff`.[4][2]
- Clarity
  - Avoid internal slang or abbreviations that others might not understand later.[^4]
  - Prefer precise descriptions like `Fix null pointer in user service` over `Bugfix`.[^8]

Conventional Commits and Typed Subjects

The Conventional Commits specification defines a structured pattern for commit messages that includes a type prefix and optional scope.[12][9]

```
<type>[optional scope]: <description>
```

```
[optional body]
[optional footer(s)]
```

- The `<type>` value classifies the change, commonly using identifiers such as `feat` (new feature), `fix` (bug fix), `docs`, `style`, `refactor`, `perf`, `test`, or `chore`.[13][9]
- The optional `scope` names the part of the system affected (for example, `feat(auth): add JWT refresh token`).[^9]
- Conventional Commits align well with semantic versioning: tools can infer breaking changes (via `BREAKING CHANGE` footers or `!` markers), minor features, and patch-level fixes from the commit history.[14][12][^9]
- This structure enables automated changelog generation, release notes, and version bumps, which is why many teams adopt it as a standard.[15][14][^9]

Process Habits That Improve Commit Messages

Good commit messages depend not only on wording but also on how commits are created.[5][1][^4]

- Make small, focused commits so that each message can accurately describe a single logical change; large, mixed commits are harder to summarize.[5][1]
- Commit relatively often during development, but use interactive rebase or squashing to clean up noisy intermediate commits before sharing them with others.[2][4]
- Review your commit history before opening a pull request to ensure message consistency and to fix typos or unclear descriptions.[3][8]
- Adhere to any project-specific conventions documented in the repository (for example, mandatory type prefixes, ticket IDs, or scopes).[8][9]

Examples

Simple, non-structured example

Good

```
Fix login redirect

Ensure users are redirected back to the page they were on after
successful login instead of always going to the dashboard.
```

- Brief subject in imperative mood (`Fix login redirect`).[6][2]
- Body explains behavior change and intent without restating the diff.[2][4]

Poor

```
fixed stuff
```

- Vague, non-descriptive, past tense, and offers no insight into what changed or why.[3][2]

Conventional Commits example

```
feat(auth): add JWT refresh tokens

Allow clients to obtain new access tokens using long-lived refresh
tokens. This reduces how often users need to log in while keeping
short access token lifetimes for security.

BREAKING CHANGE: removes legacy session cookie-based auth.
Resolves: #123
```

- Uses `feat` type with `auth` scope and an imperative description.[^9]
- Body describes motivation and high-level behavior.[4][2]
- Footer declares a breaking change and ties the commit to an issue tracker ID.[9][2]

Key Takeaways

- Use a clear structure: short subject, blank line, optional body and footer.[7][2]
- Apply consistent formatting: imperative mood, length limits, and wrapping for readability.[6][7][^2]
- Emphasize *why* the change exists and what it does, not just a restatement of the diff.[3][2][^4]
- Consider adopting a standard like Conventional Commits when you want machine-readable history for automation such as changelog generation and semantic versioning.[12][14][^9]

References

1. 12 Best Practices for Git Commit Messages - DevOps Training Institute - Clear subject lines, detailed bodies, and consistent formatting ensure that your team can collaborat…
2. How to Write a Git Commit Message - cbea.ms - 1. Separate subject from body with a blank line · 2. Limit the subject line to 50 characters · 3. Ca…
3. Good Commit Messages - Stephen Houser, Adjunct Professor of Computer Science, University of Southern Maine
4. Writing a Good Git Commit Message - Provide a Meaningful Summary – Summarize what changed at a high level; include meaningful informatio…
5. Git Commit Best Practices - GitHub Gist - Commit Message norms · 1. Make clean, single-purpose commits · 2. Commit early, commit often · 3. Wr…
6. Git commit message conventions and best practices - TheServerSide - Git commit message best practices · Limit the subject line to 50 characters. · Capitalize only the f…

7. How to Write a Git Commit Message, in 7 Steps - Chris' 7 rules of great Git commit messaging. Separate subject from body with a blank line · Limit t…
8. Best Practices for Git Commit Message | Baeldung on Ops - In this tutorial, we'll study some of the best practices for Git commit messages. First, let's under…
9. A specification for adding human and … - Conventional Commits - A specification for adding human and machine readable meaning to commit messages
10. How to Write a Good Git Commit Message | Git Best Practices - In this article, we will go over how to write good Git commit messages using GitKraken Client's CLI …
11. chris.beams.io/posts/git-commit/ details good practices … - Discussion on: A Beginner's Guide to Git — How to Write a Good Commit Message · Separate subject fro…
12. Conventional Commits Specification - Wikipedia
13. Conventional Commits: A specification for structured commit messages
14. Conventional Commits - The Conventional Commits specification is a lightweight convention on top of commit messages. It pro…
15. conventional-commits.md - GitHub Gist: instantly share code, notes, and snippets.